CSE 554 Machine Learning Systems

Lecture 6 – Transformer Implementation

Baris Kasikci

BE BOUNDLESS

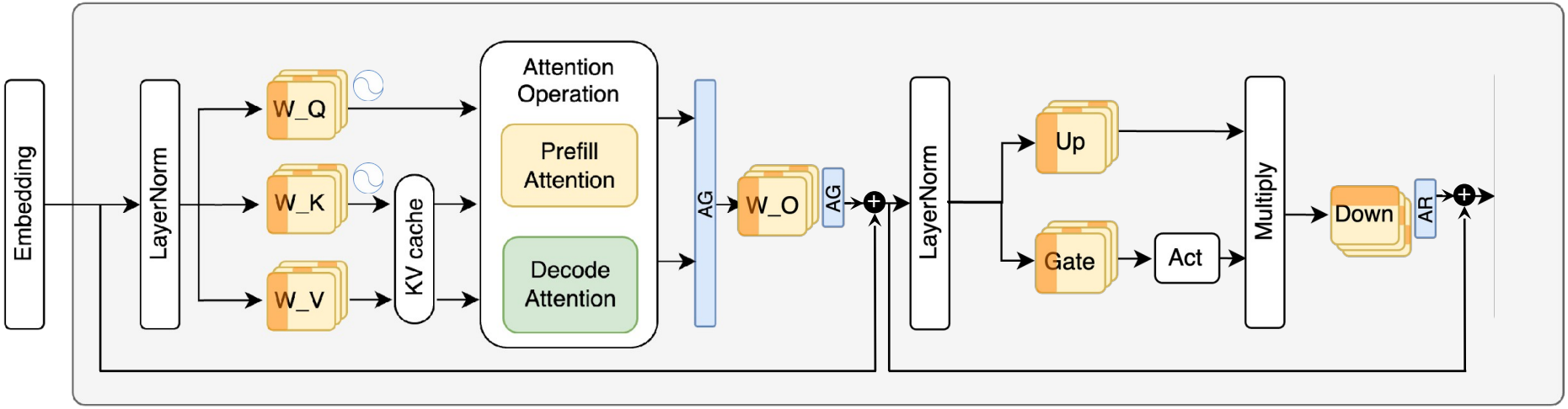
Class Overview

1. GPU architecture and programming
2. **Transformers (Architecture and Implementation)**
3. Performance modeling
4. Quantization and Sparsity
5. Memory management
6. Mixture of Experts (MoE)
7. Retrieval-Augmented Generation (RAG)
8. Batching
9. Speculative decoding
10. Parallelism
11. End-to-end serving
12. Agentic systems
13. Large-scale deployment

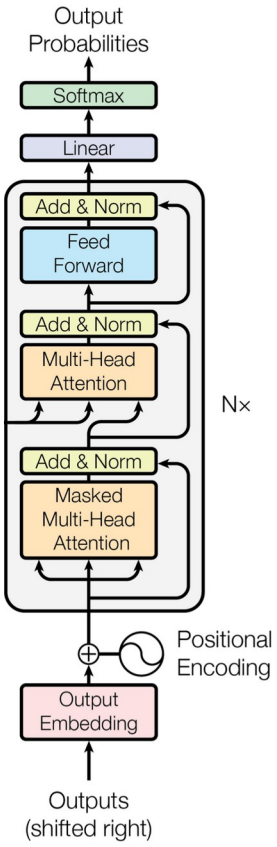
Today

- Transformer Implementation
  - Coding session

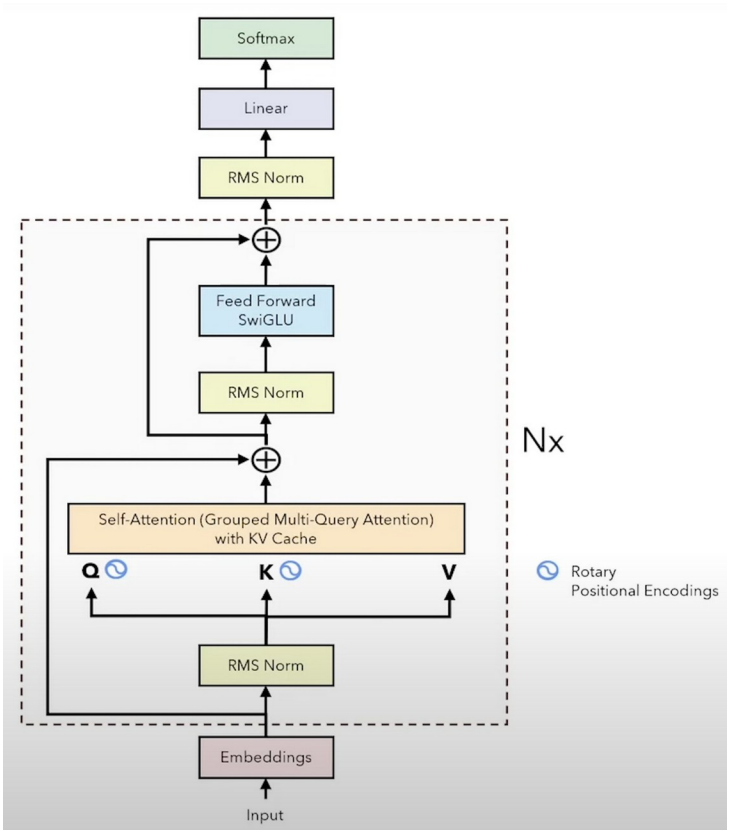
Last Time – Decoder-Only Transformer Architecture



Attention is All you Need

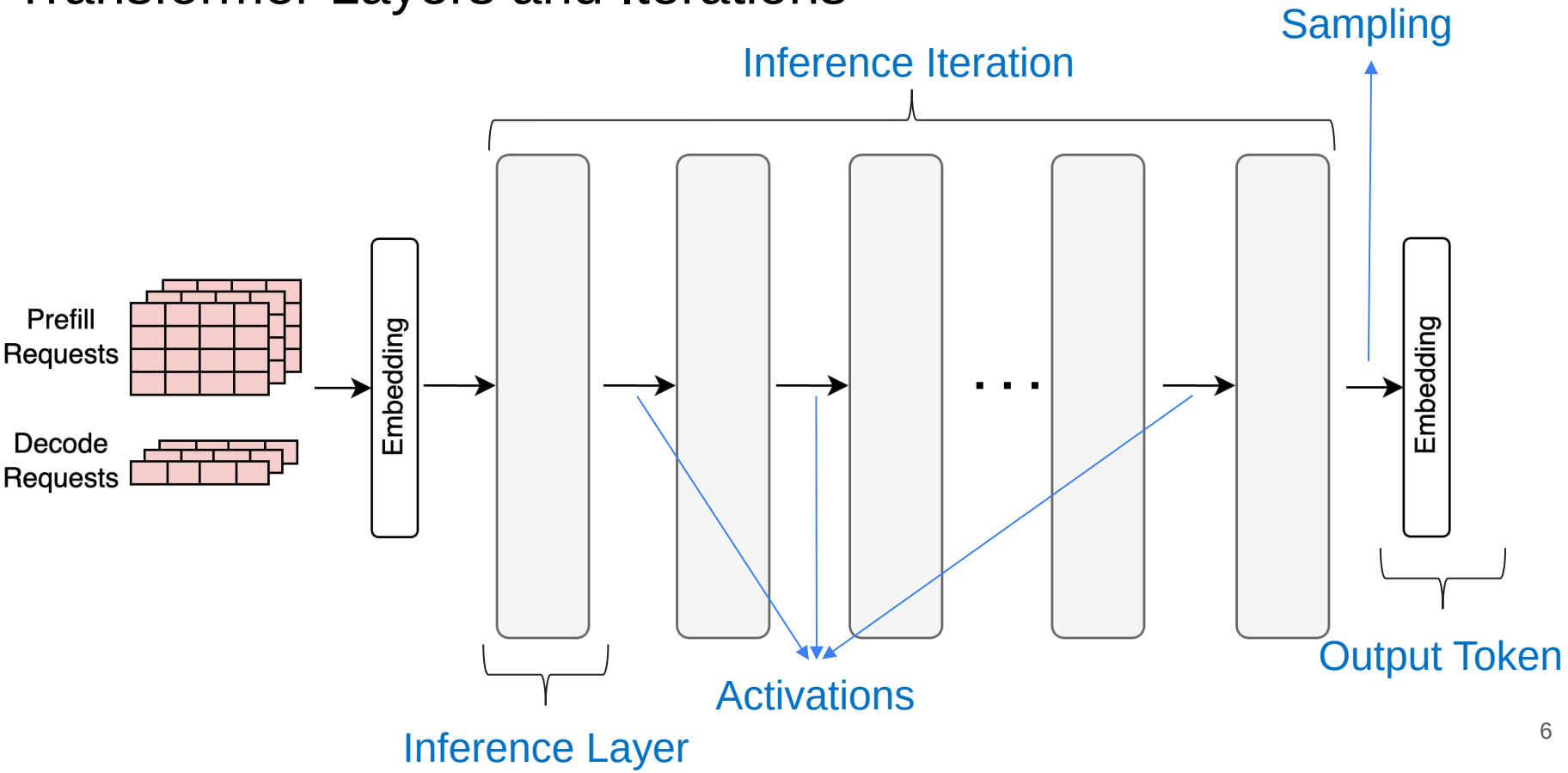


Llama Architecture*

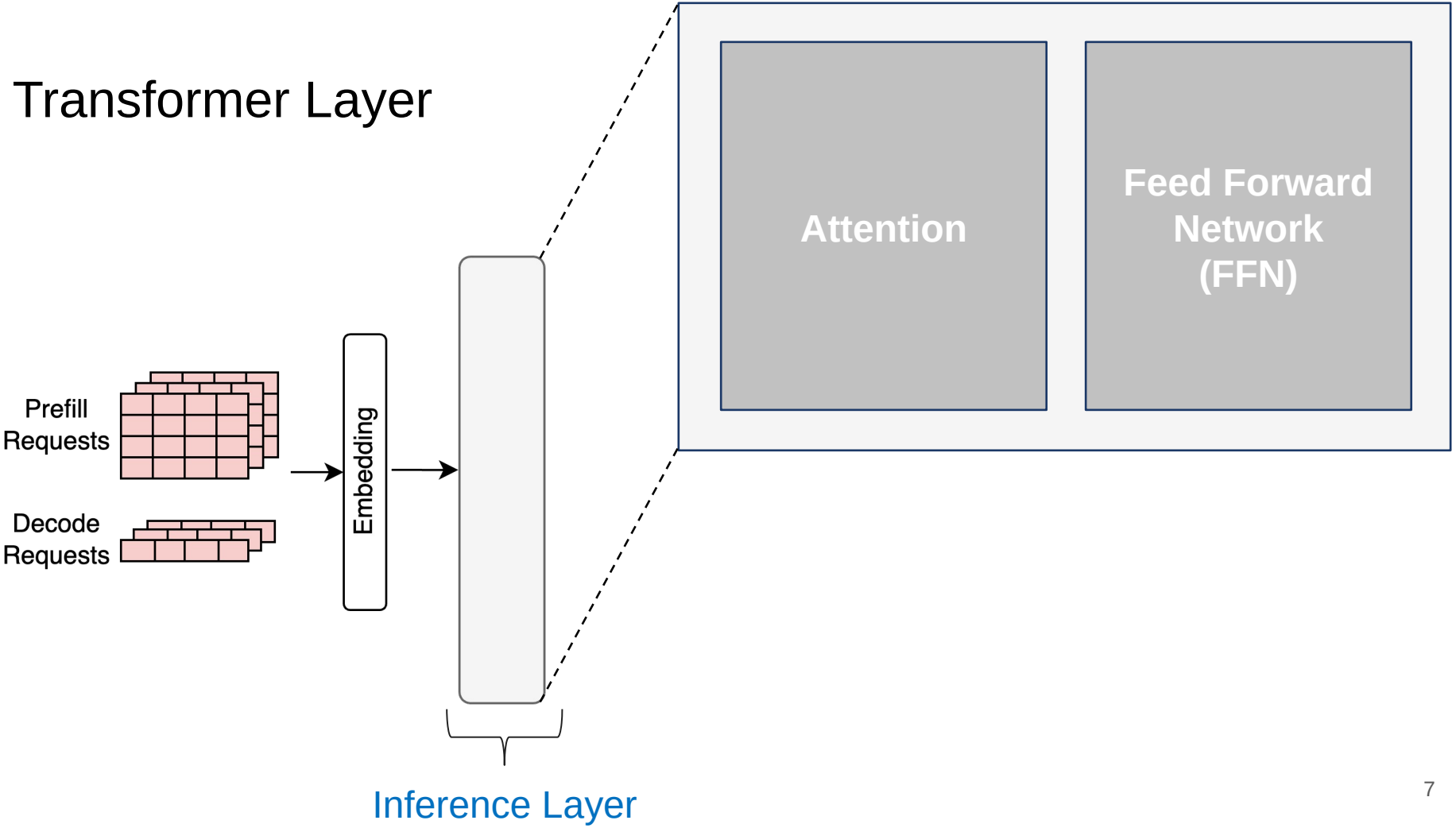


*Same as last time, lacking networking

Transformer Layers and Iterations



Transformer Layer



Indexing into embeddings

input_ids: [The, University, …]

embeddings: {… : …,

The: [0, 1, 0, 1, …],

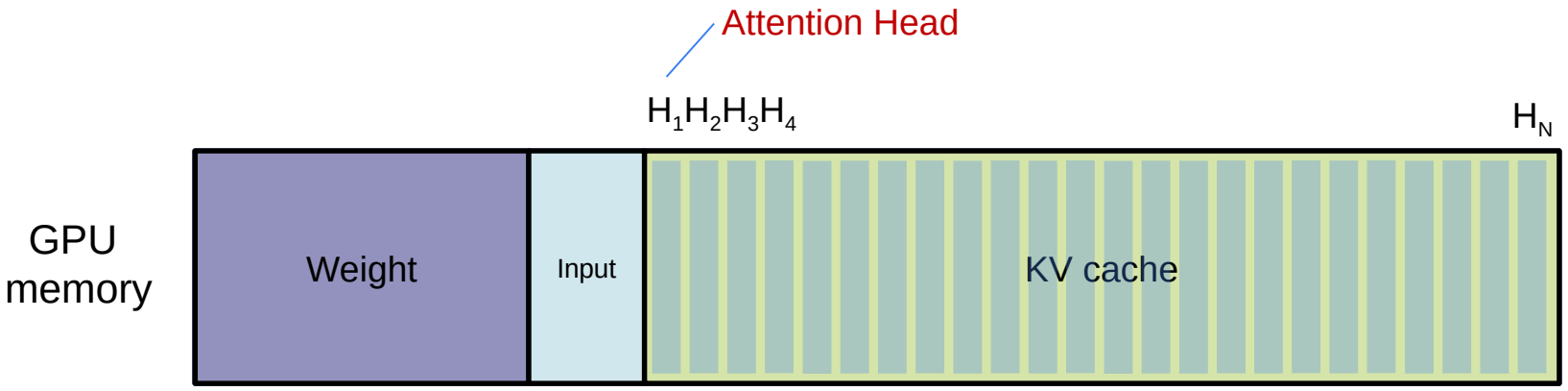
…

University: [0, 0 ,0 , …]

}

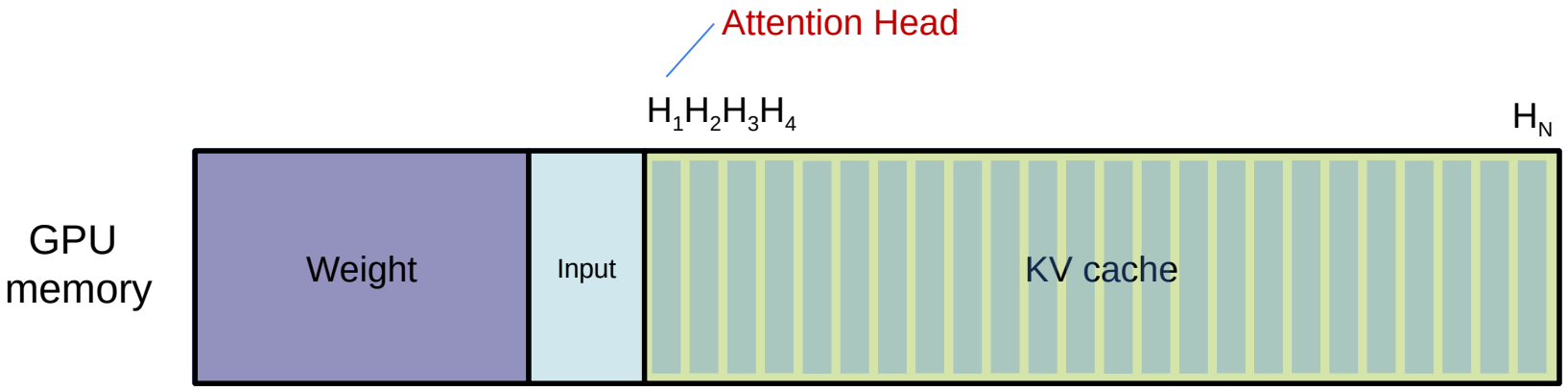
4096 elements in Llama-3

Grouped Query Attention[1]



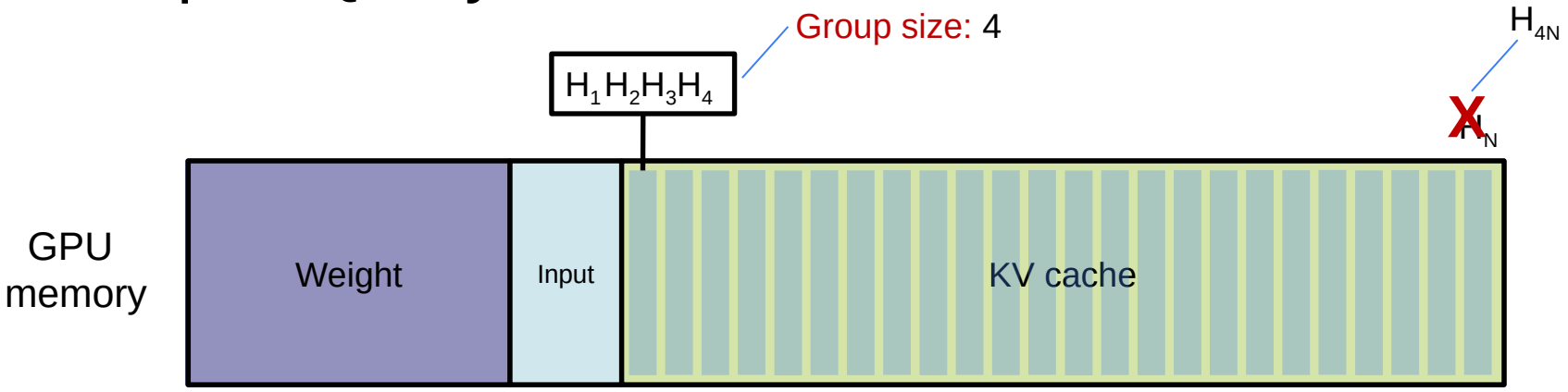
[1] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai

Grouped Query Attention[1]



[1] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai

Grouped Query Attention[1]



GQA allows increasing the batch size by a factor of the group size

[1] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, Sumit Sanghai

RMSNorm

Given a vector $\mathbf{x}$, the RMSNorm is defined as:

$$\mathrm{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\sqrt{\frac{1}{n}\sum_{i=1}^{n} x_i^2 + \epsilon}} \odot \mathbf{g}$$

where:

- $\mathbf{x}$ is the input vector of size n.
- ϵ is a small constant for numerical stability (e.g., 10^{-8}).
- $\mathbf{g}$ is a learned scaling parameter (often a vector, element-wise multiplied).

Compute sub-components of QKV for each head

(seq_len, num_qo_heads * head_dim) $\xrightarrow{\text{view}}$ (seq_len, num_qo_heads, head_dim)

View reshapes without changing axis order

Rearrange Q and K so we can do QK^T for each head

(seq_len, num_qo_heads, head_dim) $\xrightarrow{\text{permute}}$ (num_qo_heads, seq_len, head_dim)

Permute reorders axes because attention scores are computed per head

Self-Attention

$$\textbf{Attention (Q, K, V)} = \textbf{softmax}\left(\frac{\textbf{Query}\ \ \textbf{Key}^{T}}{\sqrt{d_k}}\right)\textbf{Value}$$

$$\mathrm{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

d_k is head dimension

Causal Self-Attention

$\vec{K_i} = \vec{E_i} \quad \vec{W_K_i}$

$\vec{Q_i} = \vec{E_i} \quad \vec{W_Q_i}$

K / Q	A $\vec{K_1}$	blue $\vec{K_2}$	creature $\vec{K_3}$	roamed $\vec{K_4}$	the $\vec{K_5}$	verdant $\vec{K_6}$	forest $\vec{K_7}$
A $\vec{Q_1}$	●	-∞	-∞	-∞	-∞	-∞	-∞
blue $\vec{Q_2}$	●	●	-∞	-∞	-∞	-∞	-∞
creature $\vec{Q_3}$	●	●	●	-∞	-∞	-∞	-∞
roamed $\vec{Q_4}$	●	●	●	●	-∞	-∞	-∞
the $\vec{Q_5}$	●	●	●	●	●	-∞	-∞
verdant $\vec{Q_6}$	●	●	●	●	●	●	-∞
forest $\vec{Q_7}$	●	●	●	●	●	●	●

Causal mask

Attention (Q, K, V) =

$$\text{softmax}\left(\frac{\text{Query}\ \text{Key}^T}{\sqrt{d_k}}\right)\text{Value}$$

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

xi → -∞, softmax(xi) → 0

Separating Attention Heads

```
q = [[1, 2, 3, 4, 5, 6],  # Token 1
     [7, 8, 9, 10, 11, 12]]  # Token 2
```

(seq_len, hidden_dim): (2, 6)

2 3

```
q = [[1, 2, 3, 4, 5, 6],  # Token 1
     [7, 8, 9, 10, 11, 12]]  # Token 2
```

(seq_len, num_qo_heads * head_dim): (2, 6)

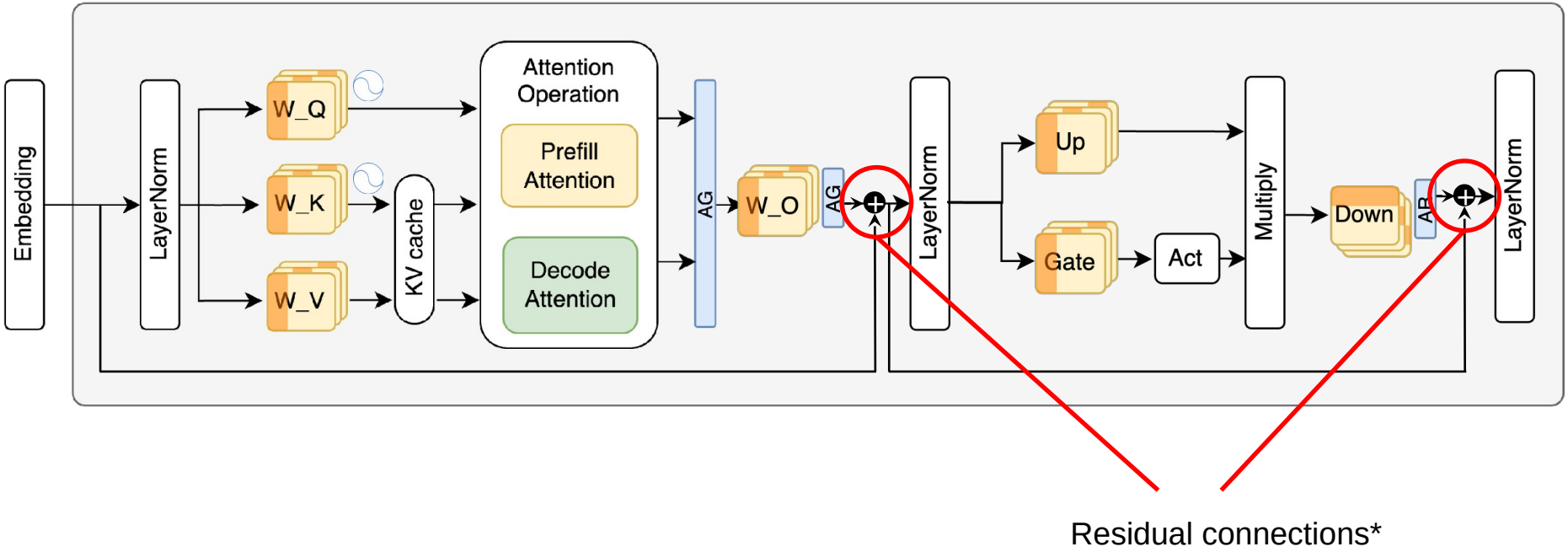
(2, 2, 3)

```
sub_q = [[[1, 2, 3],  # Head 1 for Token 1
          [4, 5, 6]],  # Head 2 for Token 1

         [[7, 8, 9],  # Head 1 for Token 2
          [10, 11, 12]]]  # Head 2 for
Token 2
```

(seq_len, num_qo_heads, head_dim)

Residual Connections



Residual connections*

*https://arxiv.org/abs/1512.03385